

RESEARCH AND IMPLEMENTATION NOTES ON

Biogeochemical Cycle Simulation

Author:
Hippolyte Audet-Lagacé

Creation date: 6th May 2026

Biogeochemical Cycle Simulation

Hippolyte Audet-Lagacé

Latest modifications: 28th May 2026

Contents

1	Introduction	2
2	Quadrilateralized Spherical Cube	3
2.1	Coordinate system	3
2.2	Face edge ownership	4
2.3	Nearest neighbors	4
2.4	Rasterizer vertex and index buffer data	5
2.5	Complexity and other implementation notes	5
3	Elevation	6
4	References	7

1 Introduction

According to all known laws of aviation, there is no way a bee should be able to fly, but bees do not care about human made laws, as they are governed by the greater laws of nature, of which some are the subject of this text. In a similar fashion, as this is a personal project of mine, I am not bound by the laws of "you can't open your text like this" and so, just as bees fly, I open my text like this.

The point of this project is to figure out how global climate works and put it into a model that approximates it well enough. What I mean by "well-enough" is arbitrary and won't be elaborated upon in the near future. I will validate the model against any relevant known dataset I can find to evaluate its accuracy and flaws as I go, and those validation will be mentioned at the end of the relevant sections.

2 Quadrilateralized Spherical Cube

The first step in this process is to model a sphere in a way that will allow for sensical solving of differential equations and equations respecting the conservation laws, and so we need a subdivision in areas of similar surface and shape, so that equations used on all areas are done under similar conditions. This problem is best solved with a Quadrilateralized Spherical Cube (QSC), the subject of this section. The methods for storing positionnal data will be described in this section and are inspired from [1] [2] [3] since their work overlaps with how I will use this sphere. A good visual representation of this model is a volleyball ¹.

2.1 Coordinate system

For simplicity's sake and without loss of generality, the spheres radius is the unit ($R = 1$). Given a right-handed orthogonal system (x, y, z) where $\hat{z}$ is the unit vector from the center to the north pole and $(\hat{x}, \hat{y})$ span the plane containing the equator, and taking (θ, ϕ) as defined by the spherical polar conventions. As reference, the position cosines (l, m, n) of a point (p) on the surface are defined as:

$$\begin{aligned}\vec{r}_p &= l\hat{x} + m\hat{y} + n\hat{z} \\ l &= \sin(\phi) \cos(\theta) \\ m &= \sin(\phi) \sin(\theta) \\ n &= \cos(\phi)\end{aligned}\tag{2.1.1}$$

Naturally, as this is going into a rasterization pipeline, we need to store Vertex positions, which conveniently coincide with the position cosines of a unit sphere. We still need to solve the problem of vertex de-duplication on the cube face edges later.

For indexing, we will need to know the 4 nearest neighbors for every vertex, and since storing them in memory and passing them to a shader everytime we need to use them would put strain on the bottleneck of modern GPU compute pipelines (memory bandwidth), an indexing scheme that allows for fast compute of the nearest neighbors is incentivized. The indexing I will used is based on the quad-tree numbering found here [3] and is visually represented in Table 1. This is also known as a Z-order curve and multiple other names and the code implementation for encoding and decoding is expanded from [4].

7								63
6								
5								
4	32							
3	10	11	14	15				31
2	8	9	12	13	24			
1	2	3	6	7	18	19	22	23
0	0	1	4	5	16	17	20	21
y/x	0	1	2	3	4	5	6	7

Table 1 – Face index relative to 2D-grid positions

In this system, every index is set by weaving together the binary values of it's (x, y) coordinates where the bits from x form the odd index bits and those from y form the even bits. Finding the grid coordinates of an index is then as simple as masking every even or odd bits to find (x, y) respectively. Finding the

1. Volleyball ball?

nearest neighbors within a face is then done by masking paired with a single bitwise operation. By taking a grid with dimensions $2^N + 1$ per face and by identifying each face with the binary values of 0-5, we can store the cells of all faces in a single 1D array by adding the face id to the left of the index stored in N bits². Along with the property just described, this is still an implicit quad-tree, or "3D binary-tree", and as such if there is ever a need to perform a search in it to find extreme values for instance, it shouldn't be very hard. This approach still leaves two problems: Face edge ownership and finding the nearest neighbors along face edges.

2.2 Face edge ownership

To avoid duplicating vertices, we need to assign each of the cube edges to a single face and in order to keep those faces squares for the indexing scheme, we need to assign exactly two touching edges to each face, which will incidentally assign them each a different corner. We then need to assign indices to the two remaining unowned corners³, these corners can be given the indices of $110...(N-1)...0$ and $110...(N-1)...1$ and will be used respectively for the north and south pole, the closest to face 0 being the north. This is represented in Figure 1 and the faces indexing follows [1, figure 4-3].

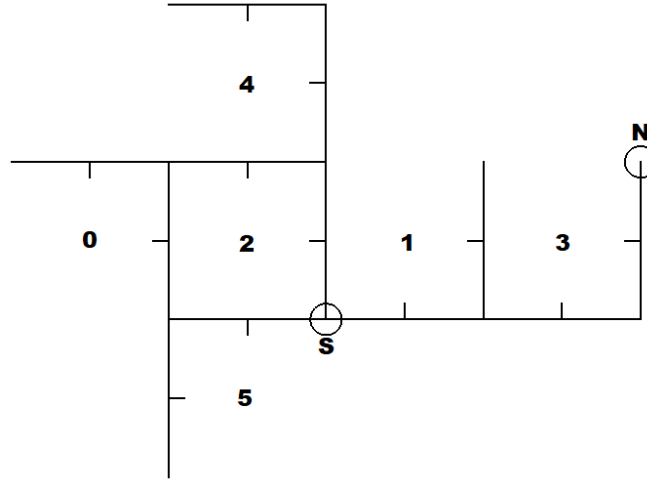


Figure 1 – Representation of edge ownership, with poles (outside of the owned edges).

2.3 Nearest neighbors

The simplest way to handle finding the nearest neighbors without complexifying the code is to handle exceptions on a case by case basis (edges, corners). The issue with this method is that whenever a warp hits the condition, it serializes the operations. However, the probability of this happening is inversely proportional to the grid size, and is pretty low for large N . For example, I've included the probabilities for $N \in \{256, 512, 1024\}$ given a warp size of 32 (2.3.1). These probabilities are of course a simplification and don't take into account compiler optimization, the logic drivers use to assemble each warp and control hazards [5, Section 3.1], but give a good enough estimation to conclude that it wouldn't be justified to implement a more complex method without noticeable performance issue tracing back to this later. As a result, the performance cost for keeping the code simple is, in my opinion, justified until further results.

2. edge ownership will leave us with an exact power of 2 as dimensions

3. We have 12 fully covered edges out of 12 and 6 covered corners out of 8, hence 2 remaining corners.

$$\begin{aligned}
\text{Formula: } P(\text{some edges}) - P(\text{all edges}) &= P(\text{branch}) & (2.3.1) \\
N = 256 : 1 - \left(1 - \frac{511}{65536}\right)^{32} - \left(\frac{511}{65536}\right)^{32} &\approx 22.16\% \\
N = 512 : 1 - \left(1 - \frac{1023}{262144}\right)^{32} - \left(\frac{1023}{262144}\right)^{32} &\approx 11.76\% \\
N = 1024 : 1 - \left(1 - \frac{2047}{1048576}\right)^{32} - \left(\frac{2047}{1048576}\right)^{32} &\approx 6.06\%
\end{aligned}$$

2.4 Rasterizer vertex and index buffer data

The vertices are made by building grids spanning the corners of each face using quaternion rotation, because the needed implementation is very lightweight, dependency free, and the equations are computationally efficient.

Now on to figuring out a way to make an index buffer for the vertex shader to properly build the faces of this "sphere". Since the index array is only built once, simple is best. Unfortunately because of the strategies used to avoid vertex duplication everything here becomes a little bit complicated. In essence, the process used here is to fill the squares within the owned vertices of faces 0, 2, 1, 3 along with the column/row 1 index past their owned edge. Because of topology two of these index sets will be shorter by 6 because of missing corners topright of faces 0 and two, which are used to store the missing squares next to the poles (these missing squares, 1 per pole, are also missing because of topology). Face 4 then only has the inner squares of its vertices to fill and face 5 has to fill the rows/column 1 vertex out from its own vertices. Of course the triangles must be built with counter-clockwise winding order so the outside faces face outside by vulkan convention.

2.5 Complexity and other implementation notes

This section doesn't have much going for it in terms of complexity as it is not computed continually, only once on initialization. Although there are divisions needed to build the quaternions, they are only built $\sqrt{n}$ times where n is the amount of vertices and even this cost is dominated by the writing operation to every vertex, and as such the overall process has a time and memory complexity of $\mathcal{O}(n)$, where n grows very fast with subdivision s : ($n = 6 * 2^{2s}$). Because of this, it is relevant to include the costs and benefits of what I would consider the reasonable range of subdivision for this program expanded by one values on each side:

Subdivision	8	9	10	11	12
Edge length	256	512	1024	2048	4096
Number of vertices	393 216	1 572 864	6 291 456	25 165 824	100 663 296
Approx distance between vertices (km)	52.18	26.09	13.04	6.52	3.26

Table 2 – Face index relative to 2D-grid positions

In words, every increment of subdivision quadruples the cost and doubles the geographic precision. Taking in consideration my resources, I will be working with $s = 9$ for development and up to $s = 11$ for testing, optimisation and producing results. The jump in memory bandwidth needed past that point is simply too high.

3 Elevation

The elevation data is extracted from the ETOPO1 global relief model using the 60 arc-second bedrock elevation dataset: <https://www.ncei.noaa.gov/products/etopo-global-relief-model>

The Rasterio python library is used to fetch 7-by-7 data grid around coordinates in a given sphere grid extracted from the sphere.rs part of the code and a combination of SciPy and NumPy methods are used to compute average elevation, standard deviation and relief for each coordinates. These computed values are then read back by sphere.rs.

This process is not automatic and requires writing the sphere grid from the rust code by calling it's write method, processing the data by fetching data from the etopo1 dataset, and then launching the code ensuring to call the sphere.rs read method.

The entire process takes a long time, very little of which coming from compute, as reading a large amount of data from a large file is simply time consuming. For a sphere of 9 subdivision, or around 1.6 million coordinates, it took me a bit over an hour of background processing. For a subdivision of 11 or more, amounting to over 25 million points, expect to wait a while.

Any .tif file for elevation data could be used, but the etopo1 dataset at 60 arc-second resolution guarantees elevation values with a maximal error of 10 meters, and by using cubic interpolation to find average elevation, the remaining error shouldn't have any notable impact on whatever model is implemented to evaluate global scale climate evolution.

This section doesn't need to be validated by me since none of this data is simulated, it is directly extracted from a dataset built on rigorously validated data, as can be seen in the above linked source's publicly available documentation.

4 References

References

- [1] F. K. Chan and E. M. O'Neill. *Feasibility Study of a Quadrilateralized Spherical Cube Earth Database*. Tech. rep. N66314-74-C-1340. Computer Science Corporation System Sciences Division, Apr. 1975.
- [2] E. M. O'Neill and R. E. Laubscher. *Extended Studies of a Quadrilateralized Spherical Cube earth Database*. Tech. rep. N00228-75-C-2329. Computer Science Corporation System Sciences Division, May 1976.
- [3] Richard A. White and Sally W. Stemwedel. “The Quadrilateralized Spherical Cude and Quad-Tree for all Sky Data”. In: *Astronomical Data Analysis Software and Systems I, A.S.P. Conference Series* 25 (1992), pp. 379–381.
- [4] Sean Eron Anderson. *Bit Twiddling Hacks*. 2005. URL: <https://graphics.stanford.edu/~seander/bithacks.html>.
- [5] Sergey Slotin. *Algorithms for Modern Hardware*. 2022. URL: <https://en.algorithmica.org/hpc/>.